

1. UI / UX & Design Layer (Easiest / Foundational)

- **Languages & Tools:** Figma (for wireframing and prototyping), HTML5, Tailwind CSS.
- **List of Work:**
 - Design wireframes for the 4 core pages: Landing Page, Donor Dashboard, Hospital/Blood Bank Portal, and the Digital Pre-Screening Form.
 - Establish a color-system (e.g., emergency red accents, high-contrast text for critical alerts).
 - Ensure mobile-first responsive design so donors can interact with alerts smoothly on their smartphones.

2. Frontend Component (Moderate)

- **Languages & Frameworks:** JavaScript / TypeScript, React.js or Next.js.
- **List of Work:**
 - Set up the project structure using React/Next.js and integrate Tailwind CSS for styling.
 - Build authentication pages (Login/Register for Donors vs. Hospital Staff with Role-Based Access Control).
 - Develop the **Donor Dashboard** with an active emergency alert inbox and map integration.
 - Develop the **Hospital Dashboard** with an emergency broadcast trigger form and live tracking response counter.
 - Build the dynamic **Pre-Screening Questionnaire** with client-side input validation (checking logic for alcohol, travel, and medications).

3. Testing & Deployment Component (Moderate to Advanced)

- **Tools:** Git/GitHub, Docker, Vercel (Frontend), Render or AWS (Backend & Database).
- **List of Work:**
 - Set up version control repository on GitHub with feature branch workflows.
 - Write basic unit tests for the pre-screening restriction logic and blood-matching algorithm.
 - Containerize the backend and database using Docker for environment consistency.
 - Deploy the frontend to Vercel/Netlify and the Python backend/PostgreSQL database to a cloud host (like Render, Railway, or AWS).

4. Messaging & Notification Component (Advanced)

- **APIs & Services:** Firebase Cloud Messaging (FCM) for push notifications, Twilio API for SMS alerts.

- **List of Work:**
 - Integrate FCM SDK for web and mobile push notifications.
 - Integrate Twilio API to dispatch high-urgency emergency text messages containing secure response links.
 - Set up a message-dispatch queue (such as Celery with Redis, or asynchronous Python tasks) to ensure alerts are broadcast instantly without blocking the server.

5. Backend Component (Advanced)

- **Languages & Frameworks:** Python (FastAPI or Django REST Framework).
- **List of Work:**
 - Initialize the Python backend server and configure API routing.
 - Implement secure user authentication using JSON Web Tokens (JWT).
 - Build the core **Matching & Filtering Algorithm** that processes incoming hospital emergency requests, filters by exact blood/plasma type compatibility, and queries the spatial database for matching donors.
 - Handle form submissions, data validation, and automated background tasks for alert distribution.

6. Database & Spatial Engine Component (Most Complex)

- **Languages & Tools:** PostgreSQL with the PostGIS extension.
- **List of Work:**
 - Design and execute the relational database schema tables:
 - Users (Donor profiles, blood types, availability status)
 - Institutions (Hospitals and blood banks)
 - Emergency_Requests (Active blood/plasma calls, units needed, urgency levels)
 - Alert_Logs & Screening_Responses (Tracking dispatches, acceptance status, and medical health restriction check results)
 - Configure GEOGRAPHY data types and spatial indexing to execute rapid geographic proximity queries (filtering for the top 10 closest donors).